NEOS TIMELINE

From starter template to structured OS shell

This PDF is the public-facing process log for the NEOS rebuild. It is written to be readable by both technical and non-technical reviewers. Each milestone explains what changed, why it changed, and what the UI looked like at that step.

Current coverage: Stage 1A (original) → SHELL-01A/01B (v2 desktop shell) → SITE-SHELL-01 (liquid glass root) → SITE-SHELL-01B (WebGL2 refraction renderer).

Project Snapshot

The project confirmed its product direction on 2026-03-14. The site root is a liquid glass channel surface. The OS desktop experience lives inside ME.EXE as an isolated channel.

- Active lane: SITE-SHELL — liquid glass channel surface.
- Phase 2 (deferred): ME-DESKTOP — OS desktop wired into ME.EXE.
- CONNECT-IMPLEMENTATION is frozen.
- The SHELL-01A/01B OS desktop code is preserved for ME-DESKTOP.

Timeline Map

1. Stage 1A (original): First shell UI pass — liquid glass channel navigation
2. SHELL-01A/01B (v2 build): OS desktop shell frame — re-scoped to ME.EXE
3. SITE-SHELL-01: Liquid glass channel surface mounted as site root
4. SITE-SHELL-01B: WebGL2 liquid glass renderer — reference-calibrated refraction, warm neutral palette (current)
5. ME-DESKTOP: OS desktop wired into ME.EXE (Phase 2)
6. Other channels: YOU.EXE, THIRD.EXE (Phase 3)

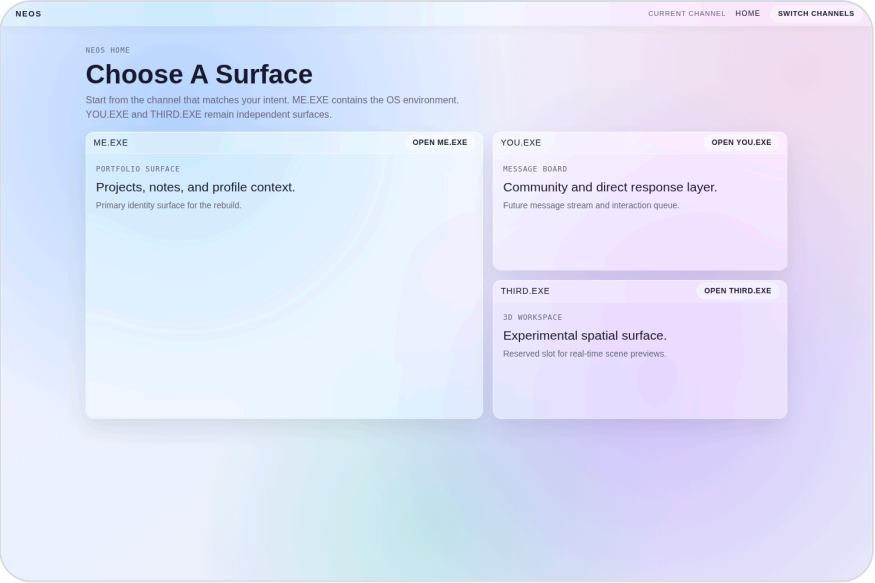
STAGE 1A — ORIGINAL

First Shell UI Pass

This milestone strengthened the HOME landing composition while preserving the scoped ME.EXE OS boundary.

HOME presented a hero-led channel surface layout, and channel switching moved to a secondary quick-switch overlay. ME.EXE owned taskbar and launcher locally.

In plain English: landing felt deliberate and substantial without collapsing into a tile-browser or smart-TV style menu.



Stage 1A: HOME channel navigation surface with hero composition and secondary channel switcher. ME.EXE owned the taskbar in its own channel.

IMPORTANT CODE — STAGE 1A

HOME Composition + Secondary Switcher

```
export function SiteShell(): ReactElement {
  return (
    <main className="neosShell">
      <div className="wallpaper" aria-hidden="true" />
      <ChannelBar ... onOpenSwitcher={() => setSwitcherOpen(true)} />
      <div className="channelViewport">
        <ChannelView channel={activeChannel} onNavigate=
{setActiveChannel} />
      </div>
      {switcherOpen ? <ChannelSwitcher ... /> : null}
      {bootVisible ? <BootOverlay /> : null}
    </main>
  );
}
```

This kept channel switching available but secondary, while preserving ME.EXE as the contained OS environment.

SHELL-01A — V2 REWRITE

Desktop Shell Frame + Boot Gate + Static Taskbar

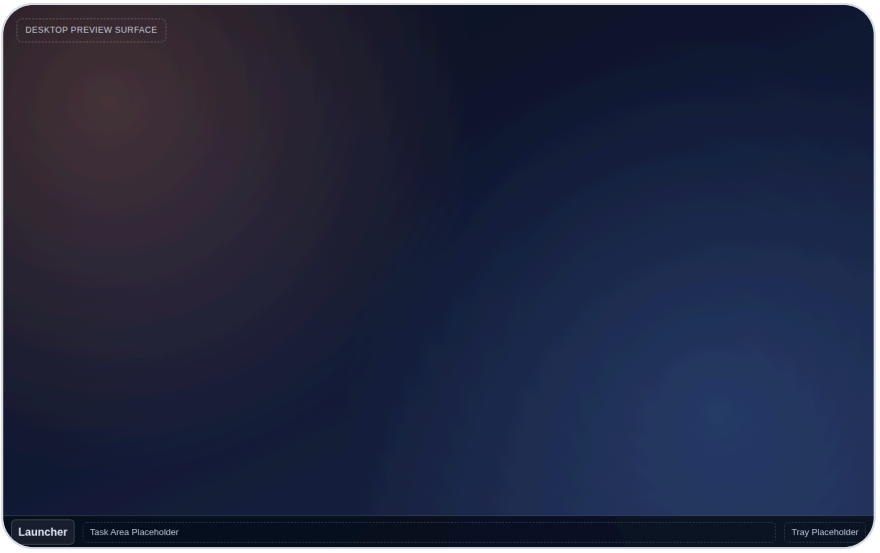
After the v2 workflow reset, the project moved to a contract-first desktop OS rewrite. SHELL-01A is the first milestone of the new architecture: a proper desktop shell frame with correct layer ordering, a boot gate, and a static taskbar.

This is a deliberate reset — not a visual upgrade of Stage 1A, but a structural rewrite with a new architecture. The channel navigation from Stage 1A is preserved in the timeline as history.

What this established

- A stable desktop root in `Desktop.tsx` with spec-correct layer order.
- A contract-shaped desktop store implementing the full `DesktopStore` interface.
- A dedicated `WindowLayer` mount surface with no runtime logic.
- A non-blocking boot gate that does not wait on VFS readiness.
- `Z_LAYER` and `SHELL_Z_INDEX` constants locked in code.

Boot sequence: a fullscreen terminal-style overlay that dismisses on click or keypress. Fires immediately on load without blocking VFS or app services.



Desktop shell after boot is dismissed: gradient wallpaper, a "Desktop Preview Surface" label marking the future icon layer, and a static taskbar with Launcher, Task Area, and Tray placeholder zones.

IMPORTANT CODE — SHELL-01A

Desktop Layer Stack

```
export function Desktop(): JSX.Element {
  const bootComplete = useDesktopStore((store) => store.bootComplete);

  return (
    <main aria-label="Desktop shell" style={desktopStyle}>
      <Wallpaper />
      <div aria-hidden="true" style={previewSurfaceStyle}>
        <span style={previewBadgeStyle}>Desktop Preview Surface</span>
      </div>
      <WindowLayer />
      <Taskbar />
      {!bootComplete ? <BootSequence /> : null}
    </main>
  );
}
```

This stack defines the layer order in code: wallpaper at the base, preview surface, window mount point, taskbar above, and boot overlay on top. Every z-index is driven by named constants, not magic numbers.

SITE-SHELL-01

Liquid Glass Channel Surface Root

This milestone rewired the app root from OS desktop boot to a light, transparent channel surface.

`src/main.tsx` now mounts `SiteShell` from `src/site/`. The site root shows ME.EXE, YOU.EXE, and THIRD.EXE entries as placeholder channel panels, while runtime internals remain isolated and deferred.

SITE-SHELL-01 entry overlay on the light liquid glass site shell

Site entry overlay: lightweight branded entry moment, distinct from OS boot sequence.

SITE-SHELL-01 channel surface with ME.EXE, YOU.EXE, and THIRD.EXE entries

Liquid glass channel surface at root level, with isolated channel entry cards and no site-level desktop runtime.

IMPORTANT CODE — SITE-SHELL-01

Root Mount Rewire

```
import { SiteShell } from './site/SiteShell';

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <SiteShell />
  </React.StrictMode>
);
```

This isolates desktop/windowing code from the site root and restores channel-surface-first navigation as the primary entry model.

What Was Intentionally Deferred in SHELL-01A

- No launcher behavior in this milestone.
- No notification or context-menu systems yet.
- No desktop icon interactions yet.
- No live app windows inside `WindowLayer` yet.
- No CONNECT gameplay or runtime implementation.

SITE-SHELL-01B

WebGL2 Liquid Glass Renderer

A raw WebGL2 refraction renderer was added beneath the site shell panels. The glass effect is physically grounded — Snell's law refraction, Poisson-disk blur, specular rim lighting, and a convex squircle surface profile matching the archisvaze/liquid-glass reference.

Parameters were calibrated to the reference defaults: IOR 3.0, blur 2 px (nearly clear), bezel 60 px, specular 0.5. The surface profile was updated from a simple power curve to the reference's convex squircle function.

The blue colour palette was replaced with warm neutrals. Buttons became nearly-transparent glass panels with backdrop blur rather than opaque white fills.

What this established

- A single full-screen WebGL2 canvas rendering all glass regions in one shader pass.
- A glass registry and React hook for components to self-register their bounding rects.
- Zero new runtime dependencies — raw WebGL2 only.
- Visual parity with the archisvaze/liquid-glass reference implementation.
- Warm neutral colour palette (no blue) throughout the site shell.

IMPORTANT CODE — SITE-SHELL-01B

Convex Squircle Surface Profile (GLSL)

```
// Matches archisvaze/liquid-glass reference
float surfaceHeight(float t) {
  return pow(1.0 - pow(1.0 - t, 4.0), 0.25);
}
```

This surface profile produces the characteristic lens-like bulge at the glass edges with a relatively flat interior — matching the reference's natural liquid glass appearance.

Next Milestone Focus

SITE-SHELL-02 should deepen channel navigation behavior while keeping the site shell lightweight and isolated from ME desktop runtime.

- Polish channel-entry interactions and transitions.
- Keep all heavy runtime mounts deferred to channel internals.
- Maintain ME-DESKTOP code unmounted at site level.
- Prepare clean handoff to ME-DESKTOP-01 (Phase 2).